



Practice Assignment – Quarter..., 20...-20...

Course Name: Backend Development

Course Code: CSW 306

Student's Full Name:

Student ID:

Practice Assignment 8

Library Management API using Repository + IRepository

Based on the previous lab, reuse the existing APIs and database that were already built, and continue developing them in this lab.

Learning Objectives

After this lab, you will understand:

- Apply the Repository Pattern in an ASP.NET Core Web API project.
- Use the IRepository<T> interface to reuse CRUD operations.
- Connect the repository to a real database using Entity Framework Core.
- Use Dependency Injection to inject repositories into controllers.
- Build clean, maintainable APIs following good architecture principles.

Required Project Structure

LibraryApi/

```
|— Controllers/
|   |— BooksController.cs
|— Data/
|   |— ApplicationDbContext.cs
|— Models/
|   |— Book.cs
|— Repositories/
|   |— IRepository.cs
|   |— BookRepository.cs
```

Exercise 1: Create the Book Model

Requirements

- Create Book.cs inside the “Models/” folder.
- Include the following properties:
 - int Id
 - string Title
 - string Author
 - int Year
 - string Genre

Exercise 2: Ensure the ApplicationDbContext is Configured

If previous labs already created ApplicationDbContext, ensure it includes:

```
public DbSet<Book> Books { get; set; }
```

If not, create it inside the “Data/” folder.

Exercise 3: Create the IRepository<T> Interface

Requirements

- Create IRepository.cs inside “Repositories/”
- Must contain generic CRUD operations that work with EF Core:

```
public interface IRepository<T> where T : class
```

```
{
```

```
    Task<IEnumerable<T>> GetAllAsync();
```

```
    Task<T?> GetByIdAsync(int id);
```

```
    Task<T> AddAsync(T entity);
```

```
    Task<T?> UpdateAsync(int id, T entity);
```

```
    Task<bool> DeleteAsync(int id);
```

```
}
```

Exercise 4: Create BookRepository (Implement IRepository<Book>)

Requirements

- Inject ApplicationDbContext via constructor.
- Implement CRUD operations using EF Core.

- Do **NOT** use hardcoded lists.

This repository must interact directly with the database.

Example responsibilities:

- GetAllAsync() → return all books
- GetByIdAsync(id) → find by id
- AddAsync(entity) → save to DB
- UpdateAsync(id, entity) → update record
- DeleteAsync(id) → remove record

Exercise 5: Register Repository in Program.cs

Add this line in **Program.cs**:

```
builder.Services.AddScoped<IRepository<Book>, BookRepository>();
```

Meaning:

- The repository now communicates with the database via EF Core.
- Controllers will receive it through dependency injection.

Exercise 6: Create BooksController Using the Repository

Requirements

- Create BooksController inside Controllers/.
- Add attributes:
[ApiController]
[Route("api/[controller]")]
- Inject IRepository<Book> into the constructor.
- Use the repository for all database operations.

Required endpoints:

1. GET /api/books → return all books
2. GET /api/books/{id} → get book by id → return 404 if not found
3. POST /api/books → add new book to database → return the created book
4. PUT /api/books/{id} → update existing book → return updated book → return 404 if not found
5. DELETE /api/books/{id} → delete book from DB → return NoContent() → return 404 if not found